

RAPPORT DE TRAVAUX PRATIQUES

TP DCLIC — SEMAINE 3

Développement d'une application mobile Magazine Info avec
Firebase

Étudiant	Omar Epiphane
Formation	DCLIC — Développement mobile
Projet	Magazine Info Firebase
Technologies principales	Flutter / Dart / Firebase / Cloud Firestore
Dépôt GitHub	Epiphane-code/Magazine_Info_firebase

Ce rapport présente le travail réalisé dans le cadre du TP DCLIC de la semaine 3. Il décrit l'objectif du projet, l'organisation de l'application, l'intégration de Firebase, la gestion des données et les principales difficultés rencontrées.

Année de formation : 2026

1. Présentation du projet

Le projet « Magazine Info » est une application mobile développée avec Flutter. L'objectif est de proposer une application permettant de gérer et consulter des informations liées à un magazine, notamment à travers des entités comme les rédacteurs et les contenus. Pour la version de cette semaine, le projet a été orienté vers l'utilisation de Firebase comme solution de stockage et de synchronisation des données.

L'utilisation de Cloud Firestore permet de stocker les données dans une base NoSQL accessible depuis l'application. Firebase est particulièrement adapté aux applications mobiles nécessitant des échanges de données simples et rapides avec un service distant.

Dépôt du projet

Le code source est associé au dépôt GitHub : https://github.com/Epiphane-code/Magazine_Info_firebase.git.

2. Objectifs du TP

Les principaux objectifs poursuivis durant cette semaine sont :

- intégrer Firebase dans une application Flutter ;
- configurer Firebase Core et les services nécessaires au projet ;
- utiliser Cloud Firestore pour la persistance des données ;
- organiser le code en séparant l'interface, la logique et l'accès aux données ;
- mettre en œuvre une gestion des données de type CRUD (création, lecture, modification et suppression) ;
- utiliser Provider / ChangeNotifier pour gérer l'état de certaines fonctionnalités ;
- versionner le projet avec Git et le publier sur GitHub.

3. Environnement technique

Élément	Utilisation
Flutter / Dart	Développement de l'application mobile
Firebase Core	Initialisation et connexion de l'application à Firebase
Cloud Firestore	Stockage des données dans une base NoSQL distante
Provider / ChangeNotifier	Gestion de l'état et communication entre l'interface et les contrôleurs
Git / GitHub	Suivi des modifications et hébergement du code source
VS Code	Édition, exécution et débogage du projet

4. Organisation de l'application

Le projet suit une organisation visant à éviter de concentrer toute la logique dans les widgets. Les responsabilités sont réparties entre les écrans/widgets, les contrôleurs et les repositories. Cette organisation facilite la maintenance et permet de faire évoluer le projet plus facilement.

Architecture logique

Interface Flutter → Controller / ChangeNotifier → Repository → Firebase / Cloud Firestore

Les widgets Flutter sont responsables de l’affichage et des interactions utilisateur. Les controllers portent la logique de présentation et notifient l’interface lorsqu’un état change. Les repositories regroupent les opérations d’accès aux données afin de limiter le couplage entre l’interface et Firebase.

Exemple de séparation

- Widgets : affichage des formulaires, listes, boutons et écrans.
- Controllers : gestion des états et déclenchement des opérations.
- Repositories : communication avec Firestore.
- Firebase : stockage distant et synchronisation des données.

5. Intégration de Firebase et Firestore

L’intégration de Firebase constitue l’un des éléments centraux du TP. L’application doit initialiser Firebase avant d’effectuer les opérations sur les services associés. Cloud Firestore est ensuite utilisé comme base de données distante pour enregistrer les informations du projet.

Principe de fonctionnement

Lorsqu’un utilisateur effectue une action depuis l’interface, le widget transmet la demande au controller. Celui-ci appelle le repository correspondant. Le repository effectue alors l’opération Firestore : ajout, lecture, modification ou suppression. Le résultat est ensuite transmis à l’état de l’application afin de mettre à jour l’interface.

Opérations de données travaillées

- Création d’un enregistrement dans Firestore.
- Lecture des documents enregistrés.
- Modification d’un document existant.
- Suppression d’un document.
- Gestion des erreurs liées à la connexion ou à l’initialisation de Firebase.

6. Travail pratique réalisé

Au cours de la réalisation, plusieurs parties de l'application ont été travaillées afin de mettre en pratique la connexion à Firebase et la structuration du code Flutter.

Gestion des rédacteurs

Une partie importante du projet concerne la gestion des rédacteurs. Un écran d'ajout permet de saisir les informations nécessaires. La logique est confiée à un controller, tandis que le repository se charge de communiquer avec la base de données.

Utilisation de ChangeNotifierProvider

ChangeNotifierProvider est utilisé pour fournir le controller aux widgets concernés. Cela permet de séparer la logique métier de l'interface et de reconstruire les éléments nécessaires lorsqu'une modification de l'état intervient.

Navigation

La navigation entre les différentes parties de l'application est réalisée avec les mécanismes de navigation de Flutter, notamment Navigator. Cette organisation permet d'ouvrir les écrans de création ou de consultation depuis les interfaces principales.

7. Difficultés rencontrées et solutions

Difficulté	Solution / apprentissage
Configuration Firebase	Vérification des dépendances, de l'initialisation Firebase et de la configuration du
Erreur dans pubspec.yaml	Correction de la déclaration des dépendances : une URL ne doit pas être utilisée c
Connexion / lecture Firestore	Vérification de l'initialisation de Firebase et de la structure des appels au repository
LateInitializationError	Vérification de l'ordre d'initialisation des objets avant leur utilisation.
Organisation du code	Séparation progressive entre widgets, controllers et repositories.

8. Tests et vérifications

Les tests ont principalement consisté à vérifier le comportement de l'application lors des opérations sur les données et à contrôler les erreurs remontées par Flutter. Une attention particulière a été portée à l'initialisation de Firebase, à la disponibilité des dépendances et à la communication entre les controllers et les repositories.

Points de contrôle

- lancement correct de l'application ;
- initialisation de Firebase avant l'accès à Firestore ;
- affichage des écrans de gestion ;
- transmission correcte des actions de l'interface vers les controllers ;

- accès aux données via les repositories ;
- absence d'erreurs bloquantes après correction des problèmes de configuration.

9. Versionnement avec Git et GitHub

Le projet est versionné avec Git afin de suivre les modifications du code et de conserver une version distante du travail sur GitHub. Le dépôt associé au TP est « Magazine_Info_firebase » sous le compte « Epiphane-code ».

Commandes utiles utilisées dans le suivi du projet :

```
git status  
  
git add .  
git commit -m "message"  
git push origin main
```

Cette pratique permet de vérifier l'état du dépôt, d'enregistrer les changements et de synchroniser le travail local avec le dépôt distant.

10. Bilan des compétences acquises

Ce TP a permis de renforcer plusieurs compétences en développement mobile. La réalisation a notamment permis de mieux comprendre le rôle de Firebase dans une application Flutter, la logique de Cloud Firestore et l'intérêt d'une architecture séparant l'interface de l'accès aux données.

Compétences renforcées

- configuration d'un projet Flutter avec Firebase ;
- utilisation de Cloud Firestore ;
- conception d'opérations CRUD ;
- gestion d'état avec Provider / ChangeNotifier ;
- organisation du code avec controllers et repositories ;
- diagnostic et correction d'erreurs Flutter/Dart ;
- utilisation de Git et GitHub pour le versionnement.

11. Conclusion

Le TP DCLIC de la semaine 3 a permis de faire évoluer l'application Magazine Info vers une architecture connectée à Firebase. L'utilisation de Cloud Firestore apporte une solution de stockage distante et adaptée à la gestion des données de l'application. La séparation entre widgets, controllers et repositories améliore également la lisibilité et la maintenabilité du projet.

Les difficultés rencontrées, notamment autour de la configuration des dépendances, de l'initialisation de Firebase et de la gestion de l'état, ont constitué des occasions de mieux comprendre le fonctionnement interne d'une application Flutter connectée à un service backend.

Dépôt GitHub : https://github.com/Epiphane-code/Magazine_Info_firebase.git
Étudiant : Omar Epiphane